Hedwig: Adaptive Governance for LLM Coding Agents

A coding agent that learns when to pause, from your real corrections, not config files.

Tanjal Shukla · Kevin Feng · Leijie Wang · Mohammad Rostami · Amy Zhang
University of Washington · Social Futures Lab | ACM CAIS 2026 | github.com/tanjalshukla/HedwigCLI

← MORE DEVELOPER CONTROL — MORE AGENT AUTONOMY →

STATIC RULES	✦ Hedwig	MILESTONE GATING	CONFIG FLAGS	FULL AUTONOMY
Rule files	**Adaptive governance**	**Approval flows**	**Preset autonomy**	**Always proceed**
Developer writes CLAUDE.md, AGENTS.md upfront. Agent follows or ignores. *CLAUDE.md · AGENTS.md*	Oversight calibrates from real interaction traces. Decisions are legible, revokable, and repo-scoped.	Human approves at fixed checkpoints regardless of content or history. *Jules · Devin*	Autonomy level set by a global flag or product default. Cannot adapt to this repo or this developer at runtime. *Cursor auto-mode · Aider --yes*	Agent acts without human oversight. Trust is binary. *AI Scientist · background agents*

A Rule Compiler

`/rules add` classifies a natural-language rule into a **hard constraint** (CLI-enforced, cannot be overridden) or a **behavioral guideline** (retrieved into prompt). One command, two enforcement tiers, no special syntax required.

B Approval Cascade

Every file action flows through four layers in strict priority order: hard constraints → active leases → PolicyScorer → preference override. **Hard constraints are absolute.** No learned preference or scorer output can override them.

C PolicyScorer

Online logistic regression with calibrated probabilities activates after **10 real decisions**. Heuristic prior carries cold-start. Quick approvals (<5s) get ½ weight to discount shallow review without blocking flow.

Scene 1 — Governed first write

```
,___, hedwig
(O,O) global.claude-sonnet-4-6
( ) mode: balanced
-"-"-
Type a task · /oversight · /help

A hedwig /rules add "never touch production_config.py"
  ✦ rule compiled · hard constraint
  type always_deny
  scope recipe_api/production_config.py
  CLI-enforced · cannot be overridden by scorer or preferences

hedwig add a prep_time_minutes field to Recipe

B ◆ hedwig · decision · read
  ? recipe_api/models.py needs approval (no prior approvals)
  ? recipe_api/service.py (affects multiple files)
  a approve r approve · don't ask again d deny: a

◆ hedwig · decision · apply
  ± edit recipe_api/models.py
  @@ -18,3 +18,4 @@
  ingredients: list[Ingredient]
  tags: list[str]
  author: str = "anonymous"
  + prep_time_minutes: int = 0

C ? recipe_api/models.py needs approval · API/interface change
  a approve r approve · don't ask again d deny: r

✓ apply approved
· API/interface change
✓ verification passed
✓ done
hedwig /status
  ◆ hedwig · session status
  mode balanced
  decisions 2 turns · adapting to your patterns
  preferences none confirmed yet
  leases recipe_api/models.py (72h)
  8 more decisions before Hedwig adapts to your patterns
```

Scene 2 — Hypothesis bank fires

```
hedwig add tag filtering to list_recipes
… planning → apply →
?
recipe_api/routes.py needs approval
a
approve d deny: d
Optional note:
just service.py, leave routes alone
hedwig add pagination to list_recipes
… planning → apply →
?
recipe_api/routes.py, tests/ needs approval
a
approve d deny: d
Optional note:
only service.py, just the logic
hedwig add a slug field to Recipe
… planning → apply →
?
recipe_api/schema.py needs approval
a
approve d deny: d
Optional note:
just models.py

D ✦ hedwig · learning · I noticed a pattern
  You've narrowed scope on me a few times —
  want me to always check in before multi-file changes?
  3 scope-narrowing messages detected.
  Applies to future sessions in this repo.
  Save this as a rule for future sessions? (y/n) [y]:
  y
  ✦ preference saved

hedwig add a delete endpoint to recipes
… planning → apply →
  ◆ hedwig · preference matched · check-in tightened
  multi-file change detected: recipe_api/routes.py, recipe_api/service.py
  scorer said proceed · preference said check-in
  →
  check-in (oversight is strictly additive)
```

Scene 3 — Legible governance state

```
E hedwig /prefs
  ✦ hedwig · learning · preferences
  Accepted
  ✦ I'll check in before multi-file changes
  Watching for (confidence from your recent decisions)
  ▓▓▓▓▓▓▓▓▓▓ 80% reviewing changes carefully
  ▒▒▒▒▒▒▒▒▒▒ new redirecting toward better approaches

hedwig /observe weights
  ◆ hedwig · observe · how my judgment has shifted · 14 decisions · adapting to your patterns
  ▲ +0.31 prior approvals more history → more trust
  ▼ -0.58 prior denials stronger signal than prior
  ▼ -0.44 files affected multi-file risk learned
  ▼ -0.19 change size
  ▲ +0.12 change type api changes tightened
  14 decisions · learned · green ▲ auto-approve · red ▼ check-in

F hedwig /retrospective
  ◆ hedwig · session wrap-up
  8 auto-approved · 3 check-ins
  Possibly too loose:
  · recipe_api/service.py auto-approved, then you pushed back
  Want me to check in more carefully? (y/n):
  y
  ✦ Got it — I'll be more careful next time.
```

D Hypothesis Bank

Evidence accumulates with context-sensitive weighting, so traces in high-relevance contexts count more. Surfaces at **≥70% confidence over ≥3 traces**. Contradicted candidates stored as rejected evidence, never silently deleted.

E Legible State

`/prefs` shows accepted preferences and live hypothesis confidence bars. `/observe weights` shows how each feature's influence has shifted. Every governance decision is traceable to which layer fired and why.

F Regret + Retrospective

Regret tracking replays auto-approved files as negative signals when the developer pushes back. `/retrospective` shows where Hedwig was too loose or too cautious. Preferences are revokable at any time.

Approval Cascade — *every file action evaluated in strict priority order, first match wins* → ✓ proceed → ⚐ flag → ⏸ check-in

① Hard constraints
Deterministic path rules compiled from `/rules add`. Always deny / check-in / allow. Cannot be overridden.

② Active leases
Time-bounded trust grants on specific files earned by prior approvals. Files with active leases skip the scorer entirely and auto-approve without interruption.

③ PolicyScorer
Online logistic regression with calibrated probabilities; heuristic prior carries cold-start. Scores risk signals → proceed / flag / check-in.

④ Preference override
Developer-confirmed preferences from the Hypothesis Bank. Can only *tighten* the scorer's output: a proceed becomes a check-in. Never loosens. Oversight is strictly additive.

Formative Study (n=21)

52% wanted check-ins at **key milestones**, not before every edit

67% frustrated agents **ignored their correction patterns** across sessions

43% couldn't understand **why** Hedwig acted autonomously

Preferences are *implicit*: developers recognize them in corrections but can't articulate them up front. Hedwig infers from behavior.

Session Signals

Hedwig infers **coding mode** (vibe / collaborative / human-only) and **session intensity** from each session's trace, shifting thresholds before the scorer fires. Override with `/oversight`.

Pilot Evaluation

Of the check-ins a cautious developer would want, how many did the system catch?

Hedwig	1.00	caught all
Agent + rules	0.50	caught half
Agent baseline	0.00	caught none

2 tasks · 11 ops · 4 labeled as requiring check-in under a cautious developer persona.

Novel Capabilities

Bui & Evangelopoulos (2026) define five criteria for proactive oversight. **Current agents satisfy at most two. Hedwig satisfies all five.**

1. **Cost-of-interruption modeled** via PolicyScorer weights + regret tracking
2. **Stay-silent is explicit:** auto-approve is a first-class logged decision with stored rationale
3. **Feedback updates policy:** online SGD classifier updates after every developer decision
4. **Cross-context signals:** session intensity and pushback type aggregate across turns
5. **Agent-initiated check-ins:** model pauses proactively; logged separately from policy-driven decisions

Current agents check in only when an explicit rule matches: no rule, no check-in. Hedwig checks in **when learned signals indicate risk, even on paths no rule anticipated.**

Context Retrieval

Each task starts with a dynamic prompt: three categories ranked by relevance to what's being asked.

Logic notes
What was built and why, matched by semantic overlap.

Behavioral guidelines
Compiled from `/rules add`. Only task-relevant rules injected, not every rule every turn.

Feedback snippets
Prior corrections by token overlap. The agent sees its past mistakes first.

Repo-aware from session one. Corrections, decisions, and guidelines feed forward automatically.

Future Work

- **Lab study:** interruption burden, steerability, calibrated reliance
- **Richer interrupt taxonomy:** distinguish wrong file, wrong approach, poor quality, or risky timing so adaptation updates the right dimension
- **Generalization** beyond coding: docs, browser agents, infra-as-code

Try Hedwig live at our booth!

GitHub · Paper